

Counting Versus Testing the Destination: A Simulation Study of Layered Defences Against SMS OTP Flooding and Pumping

Naveed Ahmed^{a,*}

^a*Affiliation to be completed before submission, Saudi Arabia*

Abstract

Endpoints that send one-time passwords by SMS are abused to waste money (flooding) or to earn part of the termination fee (SMS pumping). We evaluate in simulation a twelve-step defence that draws trust from app attestation, signed sessions and verification outcomes, against simpler designs and 25 attacker profiles. Attacks with a signature in the request are contained at little cost to simulated users; a residential attacker with farmed CAPTCHA scores is rationed, not separated, by refusing half of first-time users. On the destination, short-window counters of SMS sends per number block, each selected at a benign service target for one traffic density, leaked 6 to 48 messages against four concentrated 20-minute pumpers at 0.06 to 0.43 points of attack-free completion; at the busiest density the default sequential tests of verification outcomes leaked 19 to 452. Under a second predeclared protocol, on seeds and shifted workloads unseen by selection, the frozen counter met its leakage and benign-cost claims, leaked no more than the tests in all but 13 of 144 cells, all against an instantly verifying carrier, and failed both service-under-attack claims when the pumper shares blocks with real users. A closed-form condition explains the result: the counter's allowance grows with the blocks and windows an attack spans while the tests' does not, so it leaks less only for short campaigns on few blocks or against carriers that game outcomes.

Keywords: SMS pumping, artificially inflated traffic, one-time password, rate limiting, sequential tests, fraud prevention

*Corresponding author.

Email address: optimistbs@gmail.com (Naveed Ahmed)

1. Introduction

Most online services verify a phone number by sending it a code by SMS, from an endpoint reachable before any account exists, at a tenth of a cent to twenty cents a message. A flooder makes the service send to numbers it does not own; a pumper sends to a range whose operator shares the termination fee, turning the service’s SMS budget into its income [1]. Twitter’s owner said in 2023 that pumping had cost the company about 60 million dollars a year [2], and Huh et al. found as many attack requests as genuine ones in three months of one large service’s logs [3].

This paper starts from such an incident on a registration flow the author operated, and from the redesign that followed: a twelve-step pipeline with attested platforms, signed sessions, number checks before a message is paid for, reputation from whether codes were entered, a five-response risk score and a hard budget. With no production logs, we ask in simulation which attacks it contains against simpler designs, at what cost in completed registrations (RQ1); which layer is responsible (RQ2); when outcome-based reputation separates an attacker from simulated users (RQ3); what rationing costs when it does not (RQ4); and what destination policies cost legitimate users, with and without an attack (RQ5).

The destination became the centre of the study, because a pumper is paid only on its partner’s ranges: under a predeclared comparison a plain counter of sends per number block leaked less than our sequential tests of code outcomes, and a second predeclared protocol mapped where that holds and fails. We contribute the incident as eight named gaps, the design and an open implementation, measurements of where reputation stops separating and what rationing depends on, a closed-form condition for when counting beats testing, and both destination-policy comparisons with their failures and collateral damage; Sections 7.1–7.4 are context for the destination results (Sections 7.5–7.7).

2. Background and related work

Revenue-share fraud. In international revenue share fraud, traffic is generated to an advertised number range and the termination fee is split along the route [4]. Sahin and Francillon’s 3.14 million test numbers from twelve providers show the range to be the fraudster’s managed asset [5].

35 *Detecting artificially inflated SMS.* Huh et al. trained gradient-boosted trees
36 on 9.4 million SMS authentication logs from nineteen countries, reaching
37 89.6% recall at 0.19% false positives on web traffic [3]. About 73% of web
38 attacks sent four or fewer requests per identity a day, defeating per-entity
39 limits, and destination signals were the strongest: about 92% zero conversion
40 for attacks against 21% for genuine traffic. Their detector needs labelled
41 traffic and acts per request on request, history and country features; our
42 policies need no labels, act per destination block and see only send counts or
43 code outcomes. Theirs was not reproduced, so the two are not ranked.

44 *Defences in practice.* Twilio’s Fraud Guard blocks regions known for pump-
45 ing [6], and its SMS Pumping Risk Score advises friction from 60 and refusal
46 above 90 [7], so graded destination action is established practice; no product
47 was benchmarked. A token bucket separates a sustained rate from a burst
48 allowance; in RFC 2697 an excess bucket fills only when the committed one
49 is full [8]. CAPTCHA solving is a labour market at about a dollar per
50 thousand [9], open where machine solvers fail [10]; client puzzles [11] were
51 not evaluated. Residential proxy addresses rotate within minutes [12], and
52 carrier-grade NAT makes address reputation on cellular networks “all but
53 infeasible” [13].

54 *Identity and change detection.* One entity can present many identities [14];
55 a signed session gives continuity, not scarcity. Our block tests are Wald’s
56 sequential tests [15] run as Page’s cumulative sums [16]. Quickest change de-
57 tection minimises delay under false-alarm constraints [17], and a CUSUM’s
58 parameters follow from the error rates it must meet [18]; windowed statistics
59 and confidence sequences [19] were not evaluated, so ‘testing’ below means
60 our conversion and speed tests only. Step 7 resembles risk-based authentica-
61 tion [20]; attacks on the code once sent are a separate problem [21, 22].

62 **3. The incident and the adapted attacker**

63 This section is motivation, not evidence: no logs from the period are in
64 our hands, permission to describe the incident is being sought, and no result
65 depends on it. The flow sent an SMS to any unregistered number. The
66 first controls, v1, were reCAPTCHA v3 on web requests, one message per
67 number per minute, five requests per address per minute, a country allow-
68 list, a body-length limit, a proxy block and per-platform caps. The returning

69 abuse used thousands of residential addresses, carried the iOS header to skip
70 the CAPTCHA, and stayed under every limit.

Table 1: The v1 controls and their gaps, recalled by the operator from the incident (Recalled; not supported by logs) or found in the design review that followed (Review).

Gap	Source	v1 assumed	How it fails
A	Recalled	A platform header says which app sent the request	Sending the header reached every exemption
B	Recalled	Five requests per address per minute bound an attacker	A fresh residential address per request
C	Recalled	A number given is worth sending to	Generated numbers cost as much as real ones
D	Review	An allowed country is a safe destination	Premium ranges inside it cost far more
E	Recalled	Per-key caps bound total spend	A wide attack under every cap
F	Review	Check-then-increment is a rate limit	Two racing instances both pass
G	Review	Bulk and test flags are internal	Neither needs stronger authentication
H	Review	Each check says which check failed	Thresholds can be probed one at a time

71 *Threat model.* The attacker wants as many messages sent as possible, or
72 sent to numbers that pay it. It buys residential proxies and human-like
73 reCAPTCHA tokens (about 0.3 cents each), mints fingerprints, generates
74 numbers and knows the thresholds. A pumper also has a colluding operator’s
75 ranges, where it may enter codes, report deliveries as failed, build verified
76 history, spread over blocks and buy challenge solutions. It cannot forge an
77 attestation, use ranges its partner does not terminate, or make a stranger
78 enter a code.

79 4. Design

80 No message is paid for before every check passes, and cheap checks run
81 first. Trust comes from evidence the client cannot forge, an attestation, a
82 server-signed session and the fate of earlier codes, though such evidence can
83 be manufactured (Section 7.8).

84 4.1. The pipeline and the risk score

85 A request passes the steps of Table 2 until one refuses it. Step 0 takes
86 the platform only from a valid App Attest or Play Integrity result. Each
87 limit and reservation is one atomic Redis operation, though the enclosing
88 steps also make separate writes (Section 6). Per-number claims and the per-
89 block counter slot are released if a later step refuses, and the hourly budget
90 is reserved only for a message that is sent; address limits and source caps
91 count requests, except that a request Step 11 challenges gives its source-cap
92 count back. Every outcome after Step 0 except the challenge returns one
93 HTTP 200 body after at least 400 ms, a floor that cannot hide longer work.

Table 2: The twelve steps and the v1 gap each targets. FB is the feedback loop.

Step	Check	Gap
0	Proxy block; platform from attestation, never from the header	A
1	Signed session bound to a fingerprint; 3 requests per session per 10 min	B
2	Limits per address, /24 and ASN, lower for hosting ASNs	B, F
3-4	reCAPTCHA v3 threshold and origin check, web only	—
5	Number intelligence: cost class, sequential numbers, liveness; reads the block’s verdict stage	C, D, G
6	Body length, excluded numbers; test bypass needs a credential	G
7	Risk score and tier: allow, delay, challenge, downgrade, block	H
8	Per-number backoff, 60 s doubling to 1 h, five a day	—
9	Per-source, platform and country caps with an adaptive multiplier	—
10	Operating mode: elevated above 80% of the hourly budget, emergency at 100%	E
11	Reserve hourly count, spend and any destination slot; write state; enqueue; uniform response	E, H
FB	Outcomes per key; destination-block tests; denylist of fine keys	—

94 Step 7 computes

$$s = \sum_i w_i x_i + w_c \frac{\max(0, \theta - r_{\min})}{\theta} + w_s f + w_m i - w_v v_n - w_f v_f, \quad (1)$$

95 where the x_i are request signals (CAPTCHA score, hosting ASN, fingerprint
96 age, session requests, country mismatch, number class), $r_{\min}$ is the lowest con-
97 version ratio over the request’s keys (address to destination block), $\theta = 0.3$,
98 $w_c = 25$, f and i flag a flooded key and one whose codes are entered within

seconds, and v_n , v_f mark a number and fingerprint with verified history; other weights are 5 to 25 [23]. Tiers start at 20, 40, 60 and 80: *allow*; *delay* (a doubling wait); *challenge* (credits -20 when solved); *downgrade* (push or WhatsApp, never SMS); *block*.

4.2. The feedback loop

A send resolves as *verified* when its code is entered, *failed* after five wrong attempts or two minutes past the delivery receipt, and *undelivered* without a receipt in sixty seconds or with a failure receipt; only the first two feed reputation. A positive receipt after a negative one reopens the send; a late verification reclassifies it, but an issued verdict stands. Each transition records its effects and its time in the same compare-and-set, and a recovery sweep finishes a dead process's work: reputation counts, block-test events and outage observations apply at the transition's time, at most once, and exactly once if the sweep runs within 20 minutes.

4.3. Destination policies

A pumper is confined to its partner's ranges but can rotate numbers and blocks within them. Destination policies are keyed on the 8-digit block (10 000 numbers), so their protection weakens as a pumper spreads.

Sequential tests. The conversion statistic assumes users verify with $p_\ell = 0.8$ and a flooder with $p_a = 0.1$: a failure adds $\ln \frac{1-p_a}{1-p_\ell} = 1.50$, a verification $\ln \frac{p_a}{p_\ell} = -2.08$. The speed statistic assumes a machine enters a verified code within five seconds with $q_a = 0.9$ and a person with $q_\ell = 0.2$, since autofill removes the keystroke [24, 25]; it adds 1.50 or -2.08 . A verdict is issued, and the statistic reset, when either exceeds $h = \ln 1000 = 6.91$. The floor $S_t = \max(-ch, S_{t-1} + \text{increment})$ sets the memory: unbounded c is Wald's test, $c = 0$ Page's cumulative sum; the default is $c = 1$. A first verdict makes the block's first-time clients pass a challenge for an hour; a second within the hour moves them off SMS, refusing those without another channel. The stage is decided in the crossing's atomic write.

Counters. A counter limits SMS sends per block in a daily or short refilling window. Step 11 reserves a slot, and decides the stage, in one atomic operation when an SMS is about to go out, so a refused or challenged request costs nothing and concurrent requests cannot skip the challenge. Past the limit a graded counter challenges first-time clients, and past twice the limit

133 moves them off SMS; a refusing counter refuses. Clients with verified history
 134 are exempt from both policies, so a code entered on a pumper’s number can
 135 buy that number an exemption. A token bucket of the same rate (burst 4
 136 or 12; its challenge-tier bucket refills independently, unlike RFC 2697’s) and
 137 the counter combined with the tests were explored as alternatives.

138 4.4. Capacity controls

139 Step 9 multiplies its caps by 0.25 to 1.5 according to a baseline job com-
 140 paring each key with its expected rate, learned from the pipeline’s hourly
 141 counters as a weekly profile or from the closed hours available; with none,
 142 the static cap stands. Verified history avoids the adaptive reduction, not the
 143 static caps or budget, and the budget bounds SMS count and spend units,
 144 not vendor or fallback costs.

145 5. Leakage estimates and an operating condition

146 Against the sequential tests, a pumper reaching B blocks at rate λ with
 147 N requests leaks about

$$L \approx \min(N, kB + \lambda\tau), \quad k = \lceil h/1.50 \rceil = 5, \quad (2)$$

148 where τ , the time a send stays unresolved, is about 2.5 minutes if the carrier
 149 never verifies and seconds if it verifies at once. It assumes no prior credit,
 150 solved challenges, exemptions or outage suspension, and predicts the first
 151 verdict, not containment. A carrier verifying a share σ of codes with human-
 152 like delay gives the conversion statistic an expected increment of $\sigma(-2.08) +$
 153 $(1 - \sigma)(1.50)$, zero at $\sigma^* = 0.42$; above it the statistic drifts away from a
 154 verdict.

155 A counter of q sends per block per window lets a pumper that spans w
 156 windows send at most

$$L_c \approx \min(N, (1 + s)qBw) \quad (3)$$

157 messages, with $s = 1$ if it solves challenges. Unlike Equation (2), this grows
 158 with the attack’s length and ignores what the carrier does with the codes.
 159 Against a carrier that never verifies, the counter leaks less than the tests only
 160 while

$$B(qw - k) < \lambda\tau, \quad (4)$$

161 and against one that games outcomes while $(1 + s)qBw < N$. A graded
 162 counter acts on the share of a block’s own sends beyond q in a window,
 163 $E[(n - q)^+]/E[n]$ for per-window sends n ; with Poisson sends at mean m and
 164 $q = 4$ that share is 3.8% at $m = q/2$ and 20% at $m = q$. Equations (3)
 165 and (4) follow from the rules; k , $\lambda\tau$ and what an excess send costs a user
 166 need the simulation.

167 6. Implementation and evaluation method

168 *Implementation.* About 4000 lines of Python implement the pipeline, feed-
 169 back loop and vendor adapters on in-memory and Redis backends. Release
 170 2.8.0 (commit 1c277a0) produced the results. Release 2.8.1 repairs four de-
 171 fects outside the simulation: a replayed effect took the recovery time, so
 172 an old delivery failure could re-enter the outage window; a late verification
 173 recovered over thirteen minutes later could leave its failure counted; a cor-
 174 recting receipt inside the timeout worker’s transition lost its timeout; and a
 175 request challenged at Step 11 kept its source-cap count. Each has a regres-
 176 sion test that fails on 2.8.0, and 102 recorded runs, three per study, replay
 177 identically under 2.8.1. Of 418 tests, sixteen run two instances on a real
 178 **redis-server**, testing specific interleavings, not all. Step 11 is still not one
 179 transaction: a crash after its reservations leaks reserved slots or resolves the
 180 send as undelivered. The implementation is a research prototype.

181 *Simulation.* The pipeline runs against scripted vendor fakes on a simulated
 182 clock. After a warm-up of ten minutes (30 or 130 for pumping), attacks
 183 run over legitimate traffic at 20 requests per minute. Simulated users verify
 184 80% of codes after a lognormal delay (median 22 s; 2 s for the 20% using
 185 autofill); 70% of numbers are reachable on WhatsApp; 90% of users complete
 186 a challenge; 20% are returning account holders with a stable fingerprint, a
 187 trusted number and verified history; a returning person without them is, to
 188 the pipeline, a first-time user, so results for returning users describe trusted
 189 identities only. Separate random streams give all scenarios at a seed the same
 190 users.

191 Behavioural parameters are assumptions, annotated with any published
 192 figure behind them, such as one provider’s global conversion rate of 68% or
 193 more [26]. Attackers (Table 4) draw pool size, rate and CAPTCHA class per
 194 seed; in a nested design (ten configurations, three seeds each) these draws
 195 account for 92% [80, 98] of the variance of the farm’s leakage. Brackets

Table 3: The studies, their seeds (three per shifted point), attack durations and results. MC: Monte Carlo.

Study	Seeds	Attack	Results
Designs; ablation, interactions	0-29; 0-9	20 min	Table 5; Figures 1, 2
Dilution; score and cap sweeps; cadence	0-4; 0-9	60; 20 min	Figures 3, 4; Section 7.4
Pumping and spread	0-9	20 (60) min	Table 6; Section 7.5
False alarms; poisoner; adaptive attackers	MC; 0-4, 0-7, 0-9	24 h; 20-70 min	Sections 7.7, 7.8
Protocol 1: matched comparison	tuning 100-102; 0-9	20 min; 24 h benign	Table 7; Figure 5
Protocol 1: robustness, C1-C5	12 points	20-60 min	Sections 7.1-7.7
Protocol 2: E1 attacked service	300-309	60 min	Table 9
E2 held-out family, K1-K5	12 points	20 min	Table 8
E3 adaptive; E4 boundary	300-309; 300-304	20-360 min	Section 7.6; Figure 6
E5 selection, every density	tuning 100-102; 300-309	20 min; 24 h benign	Section 7.6

are 95% percentile-bootstrap intervals, paired per seed for differences; they describe variation under this generator, not uncertainty about real traffic.

Two protocols and their chronology. Table 3 maps the studies. The first protocol, committed before its runs, fixed the matched comparison and a robustness study over twelve Latin-hypercube points (conversion 0.6-0.9, autofill, CAPTCHA scores, WhatsApp reachability 0.3-1.0, returning share, legitimate rate) and a held-out family (60 to 120 attack requests a minute, 50 000 to 500 000 addresses, heterogeneous legitimate traffic), with claims C1-C5 that hold if true in 90% of cells, a benchmark convention, not a confidence level. The matched comparison changes only the destination policy and, at three legitimate block densities, selects on tuning seeds the least leakage summed over four concentrated pumpers among settings at or above the *tuning target*, the default’s completion minus 0.5 points (the full sequential grid only at the busiest). The robustness study’s first run failed C1 in 32 of 108 cells because attack and no-attack runs drew different users; we fixed the pairing, a fix that followed a result, and reran every study. The second protocol (claims K1-K5) was written after the matched comparison had selected three counters and committed before its runs; six of its seed values had served other robustness points, never selection. Both ran as clean invocations (9 919 and 7 925 runs, each carrying the code hash); 300 later runs use 2.8.1.

Designs and metrics. Seven designs: **v1** (header-trusted platform, static caps), **v1-corrected** (attested platform), **budget-only** (sessions, static

caps, budget), **block-limit** (a refusing counter of five sends per block per day), **conversion-only** and **speed-only** (v2 minus one block test), and **v2**. A seed is *contained* when leakage stays at most 5% of the attack rate from some minute to the window’s end, for at least five minutes. Requests that met a verdict and never completed are a *descriptive* count. *Attributable* loss compares the same requests with the policy enforced and absent, net of the reverse, in points of offered users: on attack-free traffic it is the *benign cost*, on the attacked trace the *extra harm* relative to the attacked no-policy arm. *Degradation* compares the attacked trace with attack-free operation under the same policy.

Table 4: The 25 attacker profiles by capability, and the destination policies each faced: S, sequential tests; C, counters; B, block-limit. Names and parameters: **results/attacker_profiles.md**.

Capability	Profiles	Policies
A signature in the request (5)	One address and session; datacenter rotation abroad; a premium-rate prefix; numbers walked upward; the farm with an unattested iOS header	S, B
Looks like a person (7)	Residential bot; farmed CAPTCHA with fresh or aged fingerprints; one 48-hour-old browser profile	S, B
	Low and slow, 2 or 10 requests/min over 20 ASNs; datacenter rotation buying challenge solutions	S
Games outcomes or trust (8)	Pumpers on elevated-cost or standard ranges verifying within 1 s, or 60% after 30 s	S
	A carrier entering a code only when its block nears a verdict; one reporting every delivery failed; three trust builders, 500 identity/number pairs verifying for 10 (or 30) min, then flooding	S, C
Concentrated on the destination (5)	Four pumpers on three blocks whose carrier never verifies, verifies within 1 s or 60% after 30 s, or buys challenges; a poisoner flooding the blocks simulated users concentrate on	S, C
Spread (generated)	3 to 300 blocks, at full rate or at most the counter’s quota per block	S, C

7. Results

7.1. RQ1: what is contained, and at what cost

Datacenter rotation leaks nothing under v2 because it scores in the challenge tier and declines challenges; Step 5 refuses the premium range, and the session cap holds the single client to two messages (Table 5, Figure 1). In all 108 robustness cells these attacks leaked under 5% with completion within two points of the paired no-attack run (claim C1). The walk leaks 61 [53, 70] messages before its block’s verdict (block-limit: five), the reused profile 88 [83, 95].

Table 5: Leaked share (% , mean of per-run shares) of about 600 attack requests in twenty minutes, thirty seeds, with source caps lifted (v1, block-limit BL, v2) and on (v2, with the first-time users refused and all users completing, %). v1-corrected equals v1 except on the forged header (99.5); budget-only equals v1-corrected except on the single client (0.3) and premium pumping (94.9); conversion-only equals v2; speed-only equals v2 except on the walk (13.3; caps on 12.3, 3.8, 77.3). Aged fingerprints behave as the farm. v1 with caps on: Figure 1.

Attacker	caps lifted: leaked			caps on, v2		
	v1	BL	v2	leaked	refused	completed
Single client	5.6	0.3	0.3	0.3	1.1	79.1
Datacenter rotation	69.0	69.0	0.0	0.0	1.1	79.1
Premium pumping	99.4	1.0	0.0	0.0	1.1	79.1
Sequential numbers	99.4	1.0	11.5	10.7	3.5	77.5
Reused profile	99.5	99.5	22.5	17.4	8.6	74.2
Residential bot	37.2	37.2	37.2	29.9	16.2	69.6
Farmed CAPTCHA	99.5	99.5	99.5	45.6	54.0	45.5
Forged header	100.0	99.5	99.5	45.6	54.0	45.5

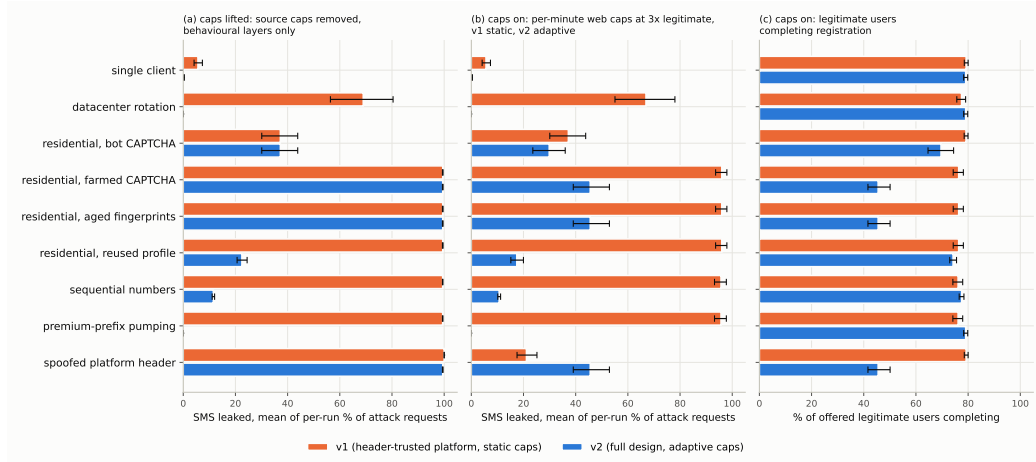


Figure 1: Leaked share of attack requests under v1 and v2: (a) source caps lifted, behavioural layers only; (b) per-minute source caps at three times the legitimate rate; (c) legitimate completion with caps on. Thirty seeds, 95% bootstrap intervals.

237 Under v1 the forged header puts the attacker in the app bucket, whose
238 static cap holds it to 100 messages; attestation removes that choice, not
239 leakage. The farmed-CAPTCHA attacker leaks 99.5% with caps lifted under
240 every design and 96% under v1 with caps on; v2's adaptive cap holds it to

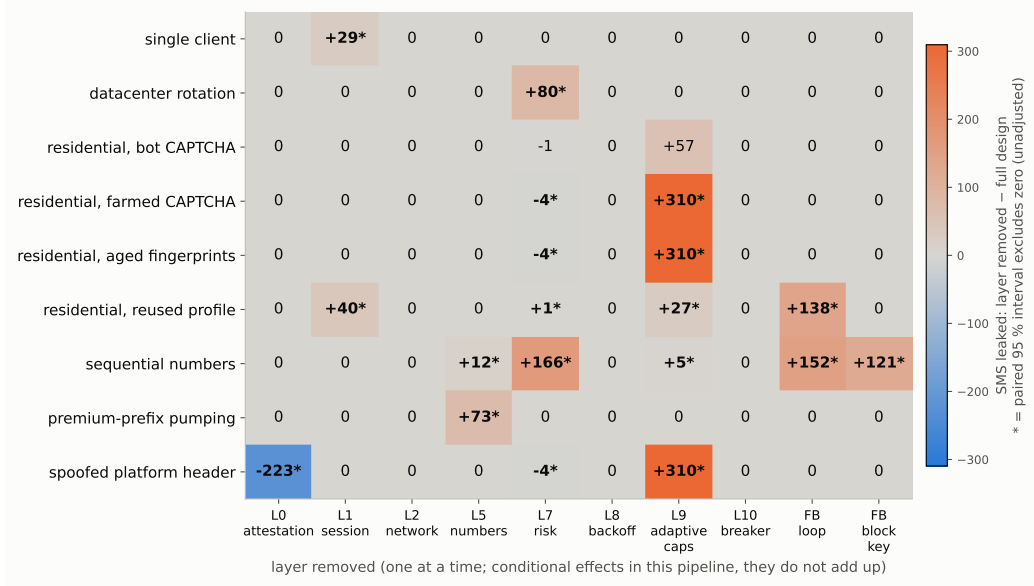


Figure 2: Ablation, caps on: leaked messages with one layer removed minus the full design, paired on ten seeds; * marks an unadjusted bootstrap interval excluding zero (19 of 90 cells). L0–L10: steps; FB: feedback loop and block key.

241 229 [215, 244] messages, 46%, by refusing 54% [47, 60] of first-time users, so
 242 45.5% complete instead of 76.4%: rationing (claim C5 held in 36 of 36 cells).

243 7.2. RQ2: conditional effects of each layer

244 Each cell of Figure 2 is a conditional effect, given all other layers. Remov-
 245 ing the adaptive caps lets the farm leak 310 [194, 421] more messages and
 246 raises completion 28.7 points; the risk engine, feedback loop and block key
 247 hold the walk by 166, 152 and 121 messages, and the loop holds the reused
 248 profile by 138. Overlapping effects do not add up: the block key acts only
 249 through the loop (interaction -121 $[-141, -101]$ on the walk), and the risk
 250 engine and the loop overlap likewise; the estimated interaction of the farm’s
 251 adaptive caps with the risk engine is small in this configuration ($+4$ $[-0.1,$
 252 $+9.5]$).

253 7.3. RQ3: where reputation stops separating

254 A farmed-CAPTCHA attacker uses an address and fingerprint once and
 255 shares its country, prefix and residential ASN with every legitimate user, so
 256 the penalty its failures earn lands on everyone.

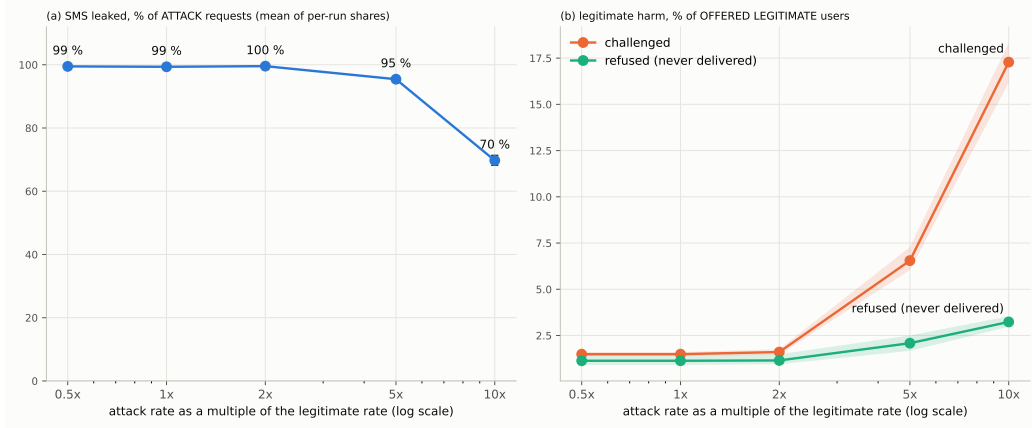


Figure 3: The farm at multiples of the legitimate rate for an hour, caps lifted, five seeds: (a) leaked share per run, intervals narrower than the markers except at 10 $\times$; (b) legitimate users challenged and refused.

Up to twice the legitimate rate, 99 to 100% of the attack leaks (Figure 3): a shared key stays above 0.3 conversion until failures outnumber legitimate sends about $(0.8 - 0.3)/0.3 \approx 1.7$ to one, and the conversion penalty alone reaches only *delay*. At ten times, 70% leaks while 17% of legitimate users are challenged and 3.2% refused (claim C2, at least 90% leaked with caps lifted, held in 34 of 36 cells). Changing the risk score one setting at a time (Figure 4b) separates no better: a fresh-fingerprint weight of 40 stops the attacker only by challenging 47% of legitimate users and refusing 8.5%, because most sign-ups are new browsers too. In the settings run, reputation separated an attacker only on keys its sends dominated [14].

7.4. RQ4: what rationing costs, and what it depends on

When separation fails, the cap is the lever (Figure 4a): the default lets 47% of the farm's requests through and refuses 41% of all legitimate users; the tightest setting, 20% and 64%. Both assume a job told the legitimate rate, an oracle; run every five minutes it lets 75 or 96% through depending on the tick's phase, and hourly 91 to 96% even of an hour-long attack. Learning the rate from the pipeline's counters instead, the deployed job leaks 193 [161, 229] messages with a clean three-week profile, 340 [239, 441] more on a cold start, which leaves the static cap, 106 [62, 151] more with a profile stale at half the rate, and 45 [10, 77] more with one poisoned at the attack's hour.

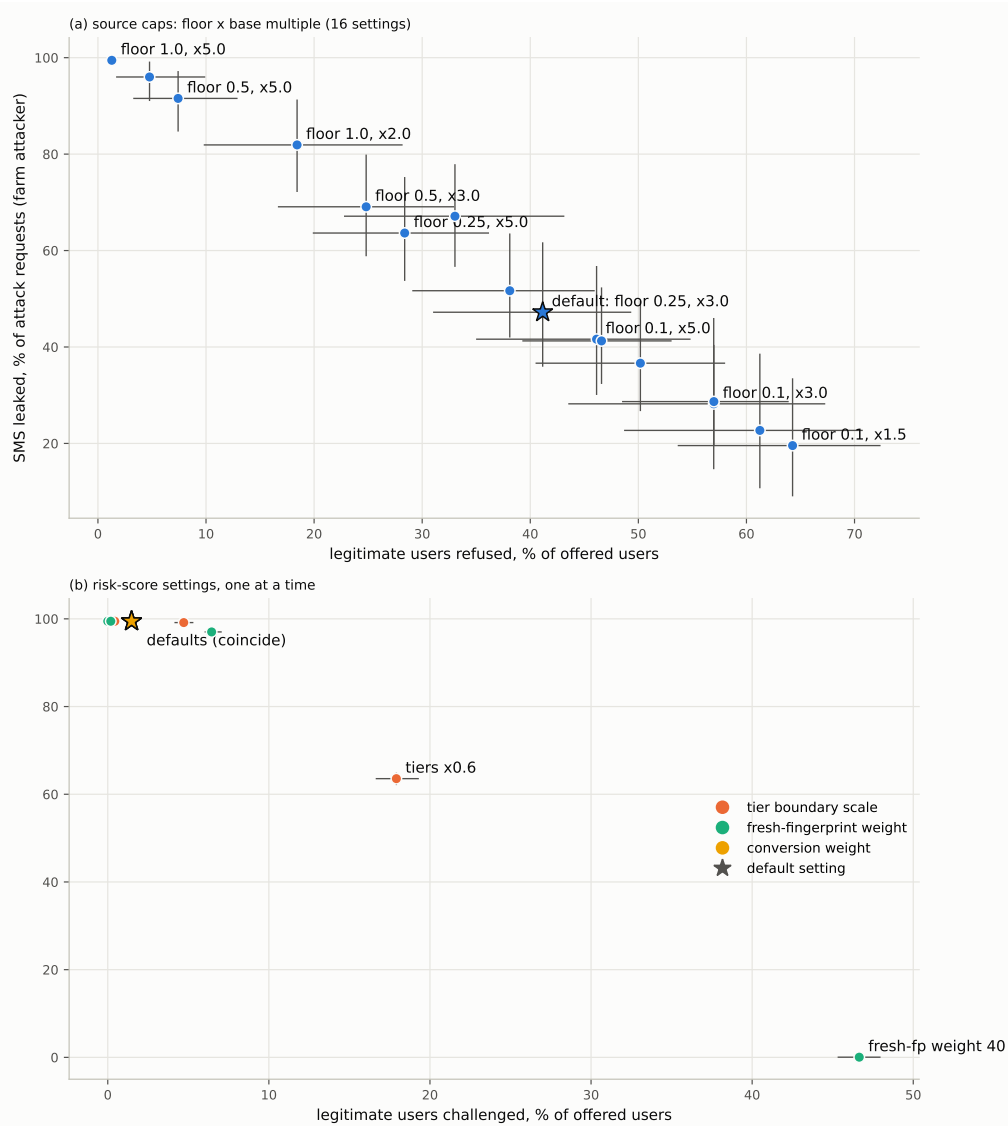


Figure 4: Farm leakage against legitimate users refused (a) and challenged (b), ten seeds: (a) cap floor and base multiple, caps on; (b) risk-score settings changed one at a time, caps lifted. Stars: defaults. Sampled points, not an optimised frontier.

Table 6: Concentrated pumpers on three blocks, caps lifted, ten seeds: messages leaked of about 574 requests in twenty minutes; seeds contained; mean minutes to containment (contained seeds) and to the first verdict (seeds with one); codes entered. A 24-hour denylist leaked 90 against the never-verifying carrier and the challenge buyer. Legitimate completion 79.1 to 79.2%.

Carrier, design	Leaked (msgs)	Contained (seeds)	Containment (min)	First verdict (min)	Codes entered
never verifies: v1	570 [419, 736]	0/10	–	–	0
v2 default	90 [71, 114]	10/10	3.5 [3.1, 3.9]	3.0	0
verifies within 1 s: v2 default	19 [17, 21]	10/10	1.4 [1.1, 1.7]	0.4	19
verifies 60% after 30 s: v2 default	452 [365, 539]	0/10	–	11.6	272
v2 default, 60 min	1067 [839, 1310]	2/10	–	–	–
buys challenges: v2 default	105 [86, 127]	10/10	4.3 [3.8, 4.9]	3.0	0

278 A pumper whose carrier never verifies is contained in all ten windows
 279 at minute 3.5 after 90 messages (Table 6); one verifying within a second is
 280 caught by the speed test at 19, and a challenge buyer leaks 105. The carrier
 281 verifying 60% with human-like delay draws its first verdict at minute 11.6 on
 282 average; no window ends contained and 452 messages leak (1067 over sixty
 283 minutes, eight runs in ten never contained). Claim C3 (containment with at
 284 most 30% leaked) held in 36 of 36 cells.

285 Across 18 spread configurations leakage follows the blocks touched: with
 286 30 blocks five to seven runs in ten are still contained, with about 240 none. In
 287 the ten unsaturated configurations Equation (2) errs by 1.6 to 7.9 messages
 288 per run (1 to 13%); saturated, it overstates by up to 91.

289 7.6. Destination policies: where the counter wins, and where it fails

Table 7: Matched comparison, first protocol, 200 blocks (144 legitimate sends per block a day), tuning seeds 100–102, evaluation seeds 0–9: twenty-minute leakage (messages) by never-verifying (NV), instant (IV), human-like (HL) and challenge-buying (CB) carriers; from 24-hour attack-free runs at 65% conversion, completion, benign cost (points), users challenged and verdict events a day (none by construction for counters). Tuning target: 63.95%.

Setting (selected as)	NV	IV	HL	CB	Compl.	Benign	Chall.	Events
none	319	566	570	405	64.13%	–	1.53%	0
sequential <i>T</i> 1000, c1 (default)	90	19	452	105	64.13%	0.01	1.62%	5.8
sequential <i>T</i> 300, c0 (selected at tuning target)	88	14	126	98	63.79%	0.34	6.24%	314.3
counter, graded, 4 per 10 min (selected)	24	24	24	48	64.07%	0.06	2.38%	0
counter, graded, 160 per day (selected daily)	319	409	408	405	64.11%	0.02	1.87%	0

Table 8: The second protocol’s claims for the principal counter (graded, 4 per 10 min) on the held-out family: 12 shifted points, held-out attack rates and pools, three seeds each, a 20-minute pumper on three blocks real users share. A claim holds if true in 90% of cells; cells share workloads and seeds and are not independent. IV: the instant verifier’s cells.

Claim	Statement	True / cells	IV	Result
K1	Leaks at most 10% of attack requests	144/144 (100%)	36/36	holds
K2	Attack-free attributable loss at most 0.5 points	36/36 (100%)	–	holds
K3	Attributable loss under attack at most 0.5 points	127/144 (88%)	32/36	fails
K4	Leaks no more than the default sequential tests	131/144 (91%)	23/36	holds pooled, not for IV
K5	Attributable loss under attack at most the default’s plus 0.25 points	93/144 (65%)	23/36	fails

290 *At a benign service target.* Under the first protocol (Table 7, Figure 5), the
291 sequential setting selected at the tuning target holds the human-like carrier
292 to 126 messages only by raising 314 false verdict events a day, and falls 0.16
293 points below the target on seeds 0–9; at the default’s false-alarm budget every
294 credit value leaks 452 or more. The graded counter of four sends per block
295 per ten minutes leaks 24 against each outcome-reading carrier (the default:
296 19 against the instant verifier) and 48 against the challenge buyer, at 0.06
297 points; counters selected at the sparser settings leak 6 to 18 at 0.15 to 0.35
298 points (up to 0.43 at 80%). With the full grid and density-specific attacks
299 (E5) the rule selects the same counters, and so does every other objective
300 we tried on the tuning runs over every setting meeting the target (worst-
301 case leakage, a first-time-loss constraint, each attacker alone, 2000 random
302 mixes), except that at 200 blocks the instant verifier alone, and 3 mixes,
303 favour a sequential setting; on fresh seeds the counter still leaked least of
304 the winners against every attacker (25, against 49 and 50, for the instant
305 verifier). On seeds 300–309 the 1000-block counter falls 0.12 points below its
306 target, yet no seed-and-conversion cell fails the paired check (the evaluation
307 default minus 0.5 points).

308 *Under a second protocol.* Frozen and run on the held-out family (Table 8),
309 the counter leaked 33, 34, 33 and 62 messages against the four carriers,
310 where the default leaked 287, 36, 344 and 287 and no policy 470 to 1857, at
311 a benign cost of 0.13 [0.09, 0.17] points, and failed both service-under-attack
312 claims: 13 of the 17 cells that broke K3 lie at the three points with the
313 most legitimate traffic (34 to 39 requests a minute), where users and pumper

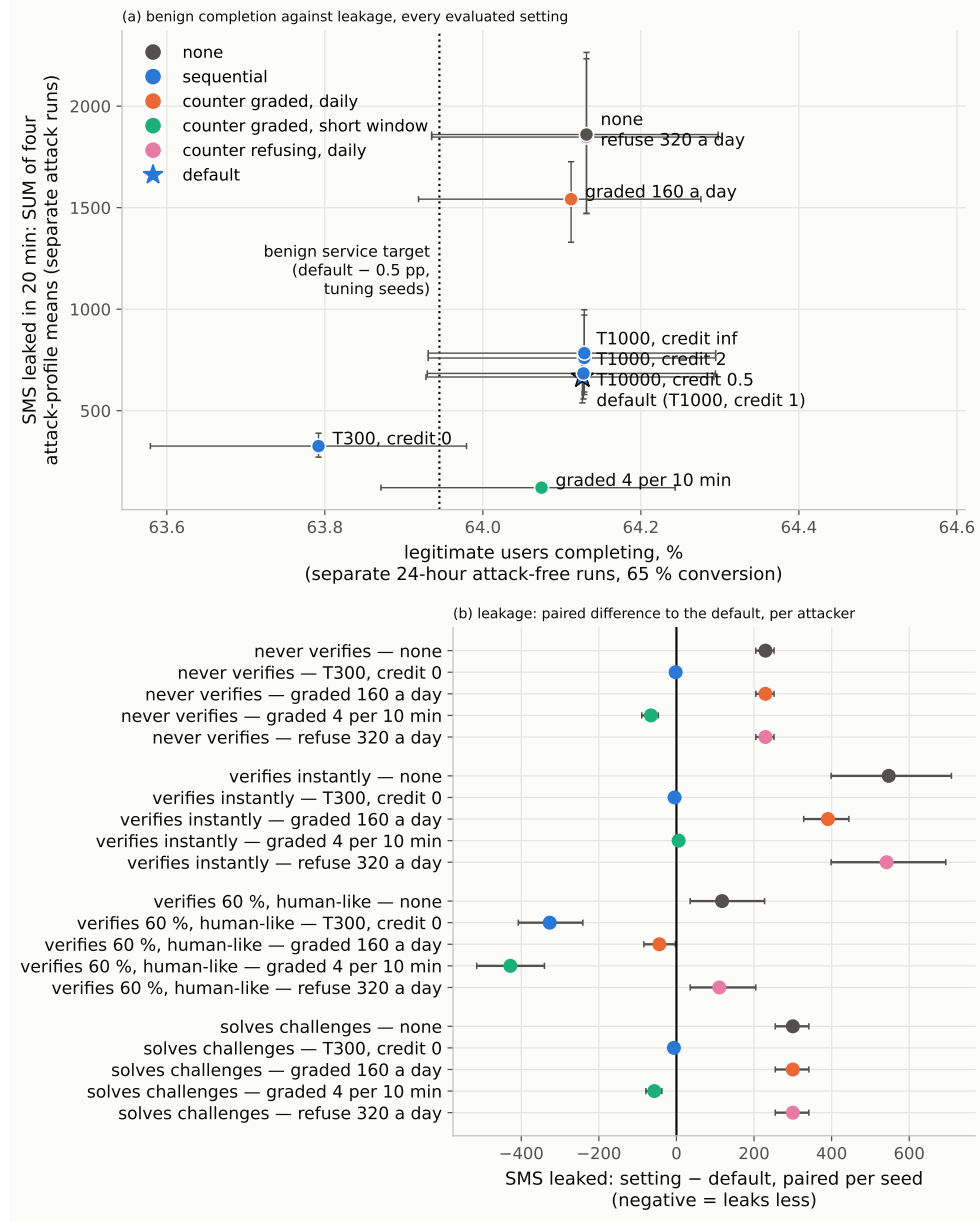


Figure 5: The matched comparison, first protocol, 200 blocks, evaluation seeds 0–9. (a) Every setting evaluated: attack-free completion against leakage summed over the four pumpers’ means, with the tuning target. (b) Per-attacker leakage differences from the default for the selected settings, paired per seed, with 95% intervals.

Table 9: Attacked service (E1): 200 blocks, a pumper on three blocks real users share, 60 minutes, ten seeds. Leakage summed and harm averaged over the four carriers, in points of all users and of the attacked blocks’ users (about 18 a run); the poisoner’s leakage and extra harm (all and first-time users). Metrics: Section 6. Negative: a gain.

Policy	Leaked	Extra harm		Degradation		Benign	Poisoner	
		all	block	all	block		leaked	harm: all / first
none	5 785	–	–	0.19	7.95	–	1 976	–
sequential, default	1 164	–0.09	–0.96	0.10	6.99	0.01	1 388	0.97 / 1.22
sequential <i>T</i> 300, <i>c</i> 0	397	0.12	1.00	0.12	8.95	0.19	871	2.45 / 3.07
counter, 4 per 10 min	403	–0.06	–3.16	0.07	4.79	0.07	1 662	0.72 / 0.91
counter + sequential	190	–0.05	–3.05	0.07	4.90	0.07	1 311	1.35 / 1.70
token bucket, burst 4	424	–0.13	–3.09	0.07	4.86	0.00	1 877	0.12 / 0.16
token bucket, burst 12	544	–0.11	–2.19	0.08	5.76	0.00	1 974	0.00 / 0.00

314 compete for one quota; K5 failed in 51 cells over 11 of 12 points. K4 held
315 in only 23 of the instant verifier’s 36 cells. The counters selected for sparser
316 traffic cost 4.7 and 5.8 points here even without an attack: the quota must
317 match the traffic.

318 Table 9 separates three questions. Against the attacked no-policy arm the
319 counter cost all users nothing and gained the attacked blocks’ users 3 points,
320 since without a destination policy the pumper exhausts their blocks and the
321 hourly budget; against attack-free operation every policy still left those users
322 5 to 9 points worse off (8 with none): no policy shields a block an attacker
323 shares. Counter and tests together leaked least against the four carriers (190
324 messages; the counter alone 403), but under the poisoner cost users 1.35
325 points (1.70 of first-time users; the counter 0.72) and leaked 1 311 messages,
326 more than the tuned sequential setting’s 871; it was explored outside the
327 frozen claims. A token bucket with a burst of 4 cost the poisoner’s users 0.12
328 points only by letting 1 877 messages through; with two untuned settings,
329 neither limiter dominates. Returning users, exempt through verified history,
330 lost nothing under these policies.

331 Attack-free (Figure 6a), the counter costs a block’s users nothing at one
332 send per ten minutes, 2.3 points at four and 5.5 at eight: about 0.12 points
333 per point of sends beyond the quota. Beyond twice it users without another
334 channel are refused, so without WhatsApp eight sends cost 13.2. A 30-
335 minute launch at three times the normal rate costs 3.7 points of all users,
336 16 of the launch blocks’ users. Over 60 minutes (Figure 6b) the counter
337 leaked 71, 240 and 679 messages on 3, 10 and 30 blocks, near its envelope
338 (72, 240, 720), and the default 110, 144 and 242, close to Equation (2) with

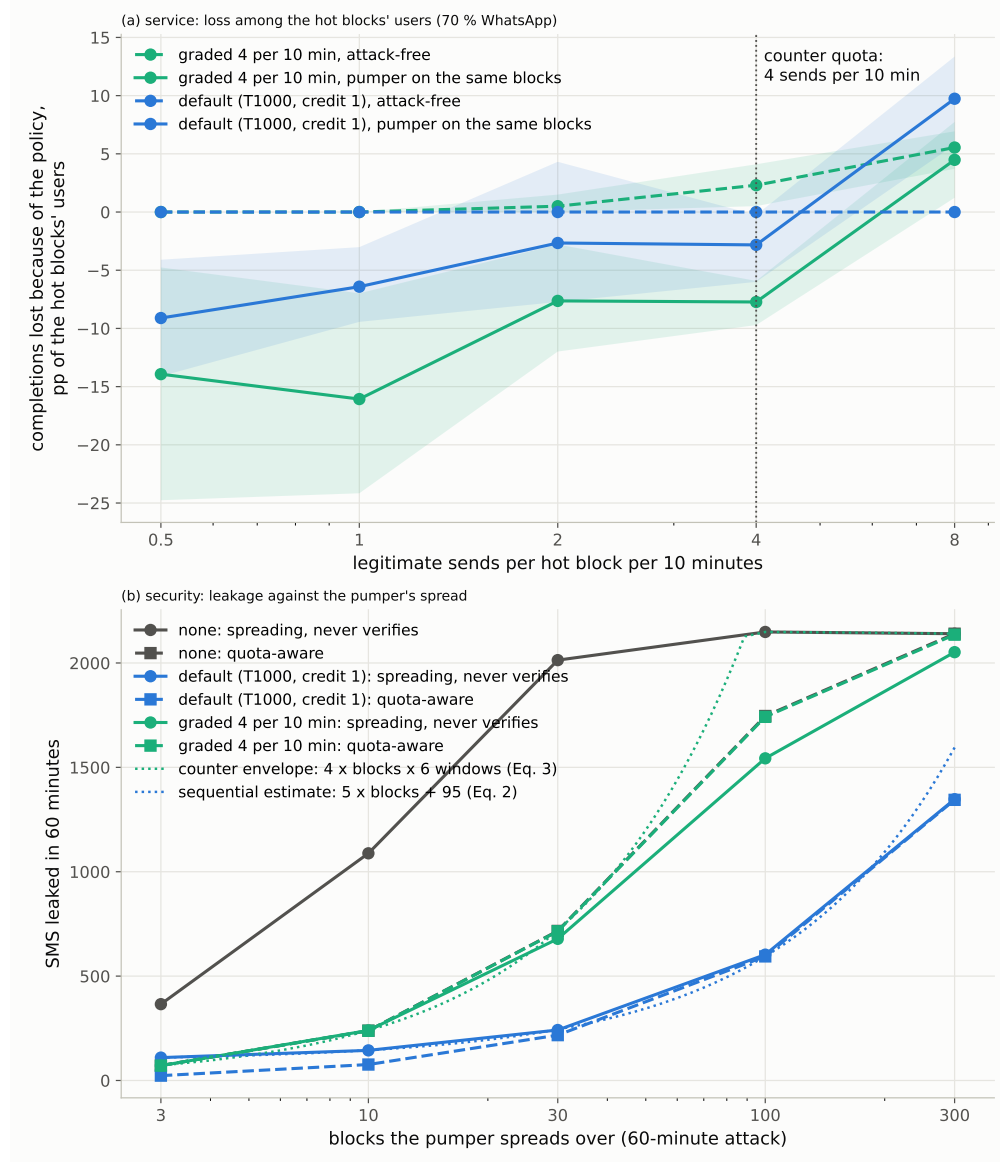


Figure 6: The counter's operating boundary (E4), five seeds; lines join sampled settings. (a) Attributable loss in points of the three hot blocks' users (70% WhatsApp reachability) against their legitimate rate, without and with a never-verifying pumper on the same blocks; bands: 95% bootstrap intervals; vertical line: the quota. (b) Mean leakage in 60 minutes against the blocks a pumper uses, at full rate (circles) or paced to the quota (squares); dotted: Equations (3) and (2) ($\lambda\tau = 95$).

339 $\lambda\tau \approx 95$; Equation (4) puts the crossover near five blocks for an hour and 30
 340 for twenty minutes, where the measured maps cross. A pumper pacing to the
 341 quota leaks what it leaks with no policy. Against adaptive attackers (E3) the
 342 counter held the threshold-aware carrier and the receipt faker to 24 messages
 343 each (665 under the default) and the concentrated trust builder to 36 (502),
 344 but not trust builders on random numbers (577, as under the default); over
 345 360 minutes the human-like carrier gained 81 messages an hour under the
 346 counter, 789 under the default.

347 7.7. *RQ5: what destination policies cost legitimate users*

348 On one block, 20 000 Monte Carlo trials per cell give the probability of
 349 a verdict within 30 legitimate sends: 0.09% [0.06, 0.14] at 80% conversion
 350 with 20% autofill, 1.9% at 65% and 19.5% at 50%. Over 24 hours the default
 351 raises 0.5 verdict events a day on 200 busy blocks at 80% conversion and 5.8
 352 at 65%, where they meet 27 of about 28 800 requests; on 1 000 blocks, 1.6
 353 and 17.9 (claim C4, at most one verdict per 1 000 requests at conversion 0.75
 354 or more, held in 18 of 18 cells). A thirty-minute outage on one prefix raises
 355 0.6 to 0.8 verdict events (send-clocked tests: 1.8).

356 A poisoner flooding for twenty minutes the twenty blocks legitimate users
 357 concentrate on (caps lifted, ten seeds) raises 16 verdict events that meet 98
 358 legitimate requests, 33 of which never complete (descriptive); against the
 359 observe-only run on the same trace, enforcement costs 12.7 [4.6, 22.9] com-
 360 pletions, 3.1% of users (6.3% without WhatsApp, 1.6% with it for all), and
 361 23.9% with a 24-hour denylist. A poisoner that stops after ten minutes leaves
 362 verdicts that meet 406.5 requests after the stop, an attributable loss of 46.2
 363 [16.9, 79.8] in that period (48.2 over the whole run). Grading limits the
 364 harm; it caps it neither at a challenge nor at the attack’s duration.

365 7.8. *Adaptive attackers, economics and overhead*

366 These scripted adaptations are not an optimised adversary; their leakages
 367 are not bounds. Against v2, a datacenter attacker buying every challenge
 368 solution leaks 55; the trust builder’s 500 pairs leak 263 while verifying, then
 369 253 of 289 flood requests; a threshold-aware carrier and a receipt faker leak
 370 570 each, drawing no verdict. Receipt-robust tests stop the receipt faker at
 371 18 but raise 30 false verdict events a day instead of 1 on blocks with a poor
 372 route.

373 If pumpers earn a share of the 0.1422 USD retail price and pay an event-
 374 level bill, they break even at a share of 0.043 under v1; under v2, at 0.271

375 (never verifies), 0.235 (buys challenges) and 0.054 (human-like), the instant
 376 verifier needing 1.310; over 360 minutes under the counter the human-like
 377 carrier needs 1.048. These rest on assumed prices, omit preparation and
 378 contracts and do not measure criminal profit; a share above one under the
 379 counter says nothing about spreading or exemption reuse.

380 On a real Redis (four vCPUs, four workers), a send costs 13.8 ms at
 381 the median and 21.1 ms at the 99th percentile. With the 400 ms floor and
 382 50 ms vendor calls, throughput is 67, 118 and 133 requests per second at
 383 concurrency 32, 64 and 128, with 0.1, 0.7 and 30% of requests over the
 384 floor. With fast vendors no pair of five outcome classes was distinguished
 385 (smallest Kolmogorov–Smirnov $p = 0.051$: no difference detected, not none;
 386 best single-threshold accuracy 53%). Under an adversarial mixture with
 387 heavy-tailed vendors at concurrency 128, 81% of sends overran the floor in
 388 server time, and a threshold on client response times fitted to one run’s 2 497
 389 sends and 22 refusals separated them with 88% balanced accuracy on that
 390 run, a resubstitution value; on three new runs (two vCPUs, nearly every send
 391 over the floor), thresholds fitted on one run scored 95 to 99.9% on the others.

392 8. Discussion

393 *Lessons.* Carriers that verify with human-like delay, fake receipts or know
 394 the thresholds defeated our outcome tests but not a short-window counter,
 395 whose allowance grows instead with the blocks and windows an attack spans.
 396 Equation (4) decides the comparison: against a never-verifying carrier the
 397 counter leaks less only for campaigns on few blocks or of short duration,
 398 against outcome-gaming carriers until its envelope reaches their volume, and
 399 it costs a block’s users roughly in proportion to their sends beyond the quota.
 400 An operator can check the condition from its own logs of per-block load and
 401 attacker spread; the cost of an excess send, the tests’ constants, competition
 402 under attack and exemption abuse come from the simulation. Counter and
 403 tests together, with a quota set from each block’s traffic, is a hypothesis for
 404 real traffic, not a recommendation.

405 *Limitations.* Everything is simulated and the calibration is assumed. Both
 406 protocols were written by the author for the author’s generator, the second
 407 knowing which counters had won. Some results follow from the generator (a
 408 perfect premium list, a datacenter attacker that declines challenges, a farm
 409 with human scores, returning users who always present verified history), and

410 selection used an unweighted sum of four scripted pumpers’ leakage and a
411 mean service target. Close rankings rest on ten or fewer seeds. Per-client
412 quotas within a block, carrier spend ceilings, bounded exemptions and other
413 sequential statistics lie outside the evaluated policies. Step 11 is not one
414 transaction, vendor adapters are untested live (App Attest enrolment only
415 on synthetic certificate chains), and the timing result holds only for the
416 observer model tested. Replaying real traffic is the experiment this paper
417 most needs.

418 *Ethics and privacy.* Destination policies challenge people for what strangers
419 sharing their block did, and fingerprints are kept for 90 days; deployment
420 under the GDPR or Saudi Arabia’s PDPL needs a lawful basis and a route
421 for refused users.

422 **9. Conclusion**

423 In this simulation the design contains attacks that carry a signature at lit-
424 tle cost to simulated users, and rations, rather than separates, a residential at-
425 tacker that looks like a person. Counting sends on the destination leaked less
426 in aggregate than our outcome tests against concentrated pumpers, though
427 not reliably against an instantly verifying carrier, and only within the condi-
428 tion of Section 5; a frozen counter failed its service claims when the attacker
429 shares blocks with real users, and loses to wide, paced and trust-building
430 pumpers.

431 **CRediT authorship contribution statement**

432 **Naveed Ahmed:** Conceptualization, Methodology, Software, Valid-
433 ation, Investigation, Writing – original draft, Writing – review & editing.

434 **Declaration of competing interest**

435 The author declares no known competing financial interests or personal
436 relationships that could have appeared to influence this work.

437 Declaration of generative AI and AI-assisted technologies in the 438 writing process

439 The author used Claude (Anthropic) for code, evaluation, drafting and
440 editing, checked numbers against the recorded results and claims against the
441 code, and takes full responsibility for the content.

442 Data availability

443 No human or real traffic data was used. Code, simulation, protocols
444 (`config/`) and every run's configuration and outcome (`results/`; per-seed
445 counter outcomes in `counter_study_per_seed.csv`, sources of headline
446 numbers in `headline_numbers.md`) are at
447 <https://github.com/nahmedpsu/OTP-DDOS>. Release 2.8.0 (tag v2.8.0,
448 commit 1c277a0) produced the results; release 2.8.1 repairs the defects of
449 Section 6 and adds the later analyses (`results/round5_analyses.md`).
450 These files are the supplementary material.

451 References

- 452 [1] MITRE Corporation, Resource hijacking: SMS pumping, technique
453 T1496.003, MITRE ATT&CK, <https://attack.mitre.org/techniques/T1496/003/>, accessed 1 October 2026.
- 455 [2] Commsrisk, Elon Musk says Twitter lost \$60mn a year because 390
456 telcos used bot accounts to pump A2P SMS, <https://commsrisk.com/elon-musk-says-twitter-lost-60mn-a-year-because-390-telcos-used-bot-accounts-to-pump-a2p-sms/>, trade press; the figure
458 is the company's own claim (2023).
- 460 [3] J. H. Huh, H. Shin, S. Ahn, H. Yi, J. Cho, T. Kim, M. Lim, N. Choi,
461 Preventing artificially inflated SMS attacks through large-scale traffic in-
462 spection, in: 34th USENIX Security Symposium, USENIX Association,
463 Seattle, WA, 2025.
- 464 [4] M. Sahin, A. Francillon, P. Gupta, M. Ahamad, SoK: Fraud in telephony
465 networks, in: 2017 IEEE European Symposium on Security and Privacy
466 (EuroS&P), IEEE, 2017. doi:10.1109/EuroSP.2017.40.

- 467 [5] M. Sahin, A. Francillon, Understanding and detecting international revenue share fraud, in: Network and Distributed System Security Symposium (NDSS), Internet Society, 2021. doi:10.14722/ndss.2021.24051.
- 468
- 469
- 470 [6] Twilio Inc., Verify fraud guard, <https://www.twilio.com/docs/verify/preventing-toll-fraud/sms-fraud-guard>, product documentation. Accessed 1 October 2026.
- 471
- 472
- 473 [7] Twilio Inc., SMS pumping risk score (Lookup v2), <https://www.twilio.com/docs/lookup/v2-api/sms-pumping-risk>, product documentation. Accessed 1 October 2026.
- 474
- 475
- 476 [8] J. Heinanen, R. Guerin, A single rate three color marker, RFC 2697, Internet Engineering Task Force (1999). doi:10.17487/RFC2697.
- 477
- 478 [9] M. Motoyama, K. Levchenko, C. Kanich, D. McCoy, G. M. Voelker, S. Savage, Re: CAPTCHAs – understanding CAPTCHA-solving services in an economic context, in: 19th USENIX Security Symposium, USENIX Association, 2010.
- 479
- 480
- 481
- 482 [10] A. Dionysiou, E. Athanasopoulos, SoK: Machine vs. machine – A systematic classification of automated machine learning-based CAPTCHA solvers, Computers & Security 97 (2020) 101947. doi:10.1016/j.cose.2020.101947.
- 483
- 484
- 485
- 486 [11] D. Dean, A. Stubblefield, Using client puzzles to protect TLS, in: 10th USENIX Security Symposium, USENIX Association, 2001.
- 487
- 488 [12] X. Mi, X. Feng, X. Liao, B. Liu, X. Wang, F. Qian, Z. Li, S. Alrizah, L. Chen, Y. Qian, Resident evil: Understanding residential IP proxy as a dark service, in: 2019 IEEE Symposium on Security and Privacy (S&P), IEEE, 2019. doi:10.1109/SP.2019.00011.
- 489
- 490
- 491
- 492 [13] P. Richter, F. Wolsing, A. Chandrasekaran, R. Padmanabhan, M. Allman, R. Bush, A. Feldmann, A multi-perspective analysis of carrier-grade NAT deployment, in: Proceedings of the 2016 ACM Internet Measurement Conference (IMC), ACM, 2016. doi:10.1145/2987443.2987474.
- 493
- 494
- 495
- 496

- 497 [14] J. R. Douceur, The Sybil attack, in: Peer-to-Peer Systems (IPTPS
498 2002), Vol. 2429 of LNCS, Springer, 2002, pp. 251–260. doi:10.100
499 7/3-540-45748-8_24.
- 500 [15] A. Wald, Sequential tests of statistical hypotheses, The Annals of Math-
501 ematical Statistics 16 (2) (1945) 117–186. doi:10.1214/aoms/1177731
502 118.
- 503 [16] E. S. Page, Continuous inspection schemes, Biometrika 41 (1/2) (1954)
504 100–115. doi:10.2307/2333009.
- 505 [17] V. V. Veeravalli, T. Banerjee, Quickest change detection,
506 arXiv:1210.5552 (book chapter) (2012).
- 507 [18] NIST/SEMATECH, CUSUM control charts, section 6.3.2.3, e-handbook
508 of statistical methods, [https://www.itl.nist.gov/div898/handbook](https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm)
509 [/pmc/section3/pmc323.htm](https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm), accessed 2 October 2026.
- 510 [19] S. R. Howard, A. Ramdas, J. McAuliffe, J. Sekhon, Time-uniform, non-
511 parametric, nonasymptotic confidence sequences, The Annals of Statis-
512 tics 49 (2) (2021) 1055–1080. doi:10.1214/20-AOS1991.
- 513 [20] S. Wiefeling, L. Lo Iacono, M. Dürmuth, Is this really you? An empirical
514 study on risk-based authentication applied in the wild, in: ICT Systems
515 Security and Privacy Protection (IFIP SEC 2019), Vol. 562 of IFIP Ad-
516 vances in Information and Communication Technology, Springer, 2019,
517 pp. 134–148. doi:10.1007/978-3-030-22312-0_10.
- 518 [21] C. Mulliner, R. Borgaonkar, P. Stewin, J.-P. Seifert, SMS-based one-
519 time passwords: Attacks and defense, in: Detection of Intrusions and
520 Malware, and Vulnerability Assessment (DIMVA 2013), Vol. 7967 of
521 LNCS, Springer, 2013, pp. 150–159. doi:10.1007/978-3-642-39235
522 -1_9.
- 523 [22] H. Siadati, T. Nguyen, P. Gupta, M. Jakobsson, N. Memon, Mind your
524 SMSes: Mitigating social engineering in second factor authentication,
525 Computers & Security 65 (2017) 14–28. doi:10.1016/j.cose.2016.
526 09.009.
- 527 [23] N. Ahmed, OTP Guard: a layered SMS validation, risk-scoring and
528 rate-limiting pipeline against OTP flood abuse (version 2.8.1), <https://github.com/nahmed/otp-guard>.

- 529 [//github.com/nahmedpsu/OTP-DDOS](https://github.com/nahmedpsu/OTP-DDOS), results produced by version 2.8.0,
530 commit 1c277a0 (2026).
- 531 [24] Google LLC, SMS retriever API overview, <https://developers.google.com/identity/sms-retriever/overview>, developer documenta-
532 tion. Accessed 1 October 2026.
- 534 [25] Web Incubator Community Group, WebOTP API, <https://github.com/WICG/WebOTP>, specification repository. Accessed 1 October 2026.
535
- 536 [26] Twilio Inc., Verify API, <https://www.twilio.com/en-us/user-authentication-identity/verify>, product page. Accessed 3 October 2026.
537